

simple_reg

March 5, 2026

```
[1]: import os

os.chdir("../")
```

```
[2]: from CensusForge import CensusAPI
from prpop_repl import DataUtils
import statsmodels.formula.api as smf
import bambi as bmb
import arviz_plots as azp
import seaborn as sns
import matplotlib.pyplot as plt
import arviz as az
import pandas as pd
import numpy as np
import polars as pl
azp.style.use("arviz-variant")

capi = CensusAPI()

dutils = DataUtils()
```

```
[3]: df = dutils.pull_dp03().filter(pl.col("state") == 72).to_pandas()
df["log_population"] = np.log(df["total_population"])
df
```

```
[3]:
```

	total_population	state	county	year	log_population
0	20295.0	72.0	9.0	2011	9.918130
1	20580.0	72.0	79.0	2011	9.932075
2	27617.0	72.0	27.0	2011	10.226187
3	14007.0	72.0	67.0	2011	9.547313
4	23742.0	72.0	105.0	2011	10.075001
...	...	...	...	...	...
1009	46109.0	72.0	145.0	2023	10.738764
1010	6922.0	72.0	147.0	2023	8.842460
1011	18147.0	72.0	149.0	2023	9.806260
1012	25723.0	72.0	151.0	2023	10.155141
1013	28929.0	72.0	153.0	2023	10.272600

[1014 rows x 5 columns]

```
[4]: # freq regersion log
results = smf.ols('log_population ~ year', data=df).fit()
print(results.summary())
```

```

                        OLS Regression Results
=====
Dep. Variable:          log_population    R-squared:                0.001
Model:                  OLS              Adj. R-squared:         -0.000
Method:                 Least Squares     F-statistic:             0.6461
Date:                  Thu, 05 Mar 2026   Prob (F-statistic):       0.422
Time:                  19:04:34          Log-Likelihood:          -1194.5
No. Observations:      1014             AIC:                    2393.
Df Residuals:          1012             BIC:                    2403.
Df Model:               1
Covariance Type:       nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
Intercept	20.8794	13.317	1.568	0.117	-5.254	47.012
year	-0.0053	0.007	-0.804	0.422	-0.018	0.008

```
=====
Omnibus:                 83.888    Durbin-Watson:           1.927
Prob(Omnibus):            0.000    Jarque-Bera (JB):        391.618
Skew:                    -0.202    Prob(JB):                9.15e-86
Kurtosis:                 6.018    Cond. No.                1.09e+06
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 1.09e+06. This might indicate that there are strong multicollinearity or other numerical problems.

```
[5]: # bayes regression log
model = bmb.Model(
    "np.log(total_population) ~ year + C(county)",
    df
)
results = model.fit()
```

Initializing NUTS using jitter+adapt_diag...

Multiprocess sampling (4 chains in 4 jobs)

NUTS: [sigma, Intercept, year, C(county)]

Output()

Sampling 4 chains for 1_000 tune and 1_000 draw iterations (4_000 + 4_000 draws total) took 3 seconds.

The rhat statistic is larger than 1.01 for some parameters. This indicates problems during sampling. See <https://arxiv.org/abs/1903.08008> for details
The effective sample size per chain is smaller than 100 for some parameters. A higher number is needed for reliable rhat and ess computation. See <https://arxiv.org/abs/1903.08008> for details

```
[6]: year_samples = results.posterior['year'].values.flatten() # This is assuming
    ↪ 'year' is the parameter you're interested in

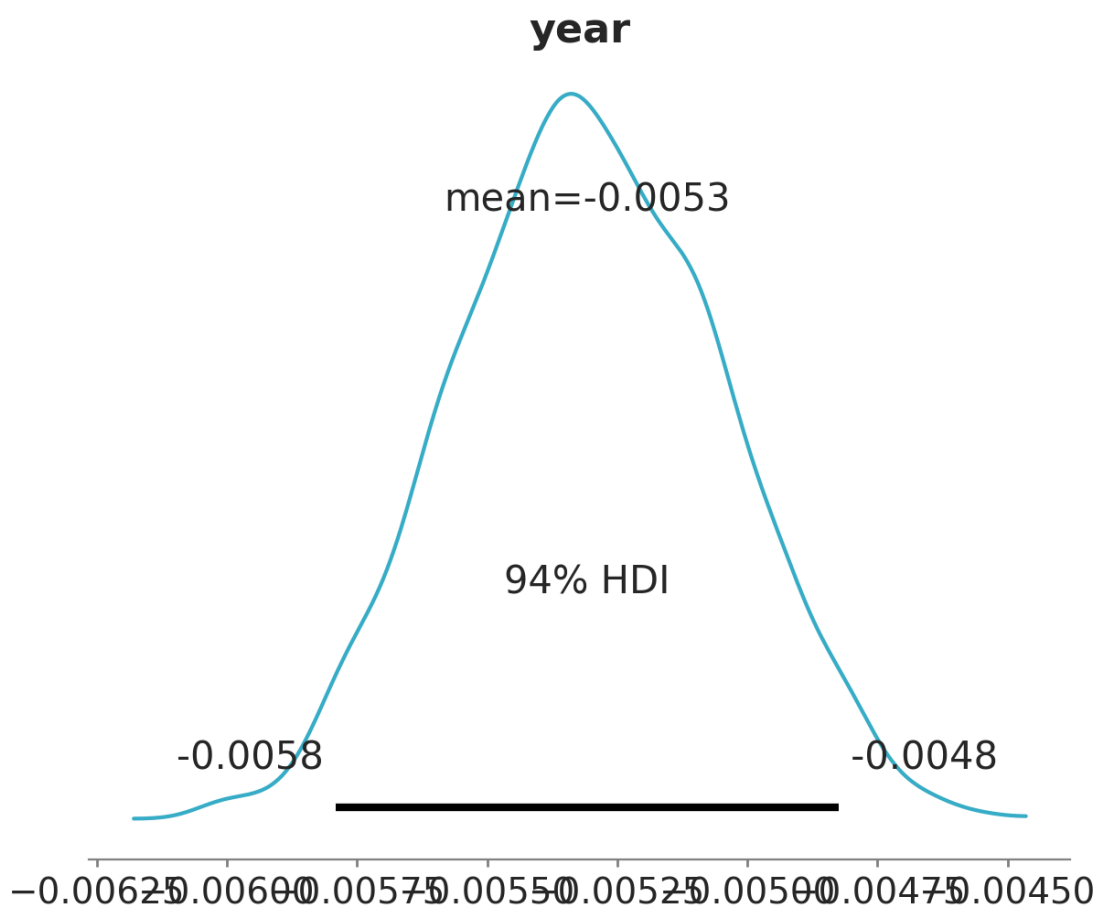
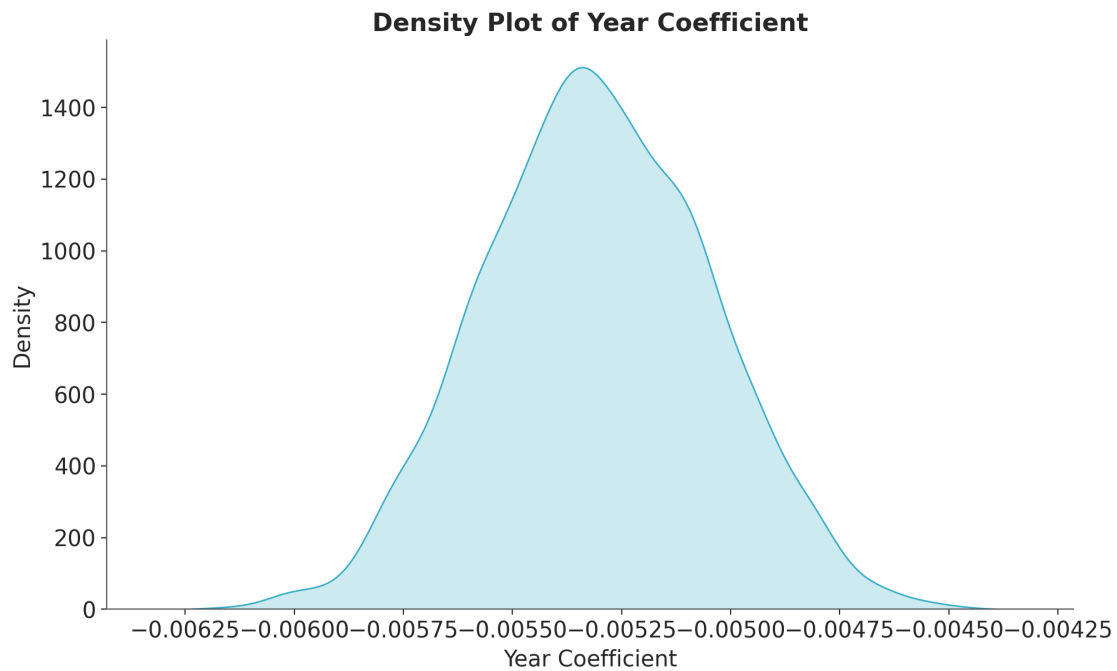
# Plot the density for the 'year' coefficient using seaborn
plt.figure(figsize=(10, 6))
sns.kdeplot(year_samples, shade=True)
plt.title('Density Plot of Year Coefficient')
plt.xlabel('Year Coefficient')
plt.ylabel('Density')
plt.show()

# Alternatively, using arviz to plot the posterior of the 'year' coefficient
az.plot_posterior(results, var_names=['year'])
plt.show()
```

/tmp/ipykernel_39973/2896428637.py:5: FutureWarning:

`shade` is now deprecated in favor of `fill`; setting `fill=True`.
This will become an error in seaborn v0.14.0; please update your code.

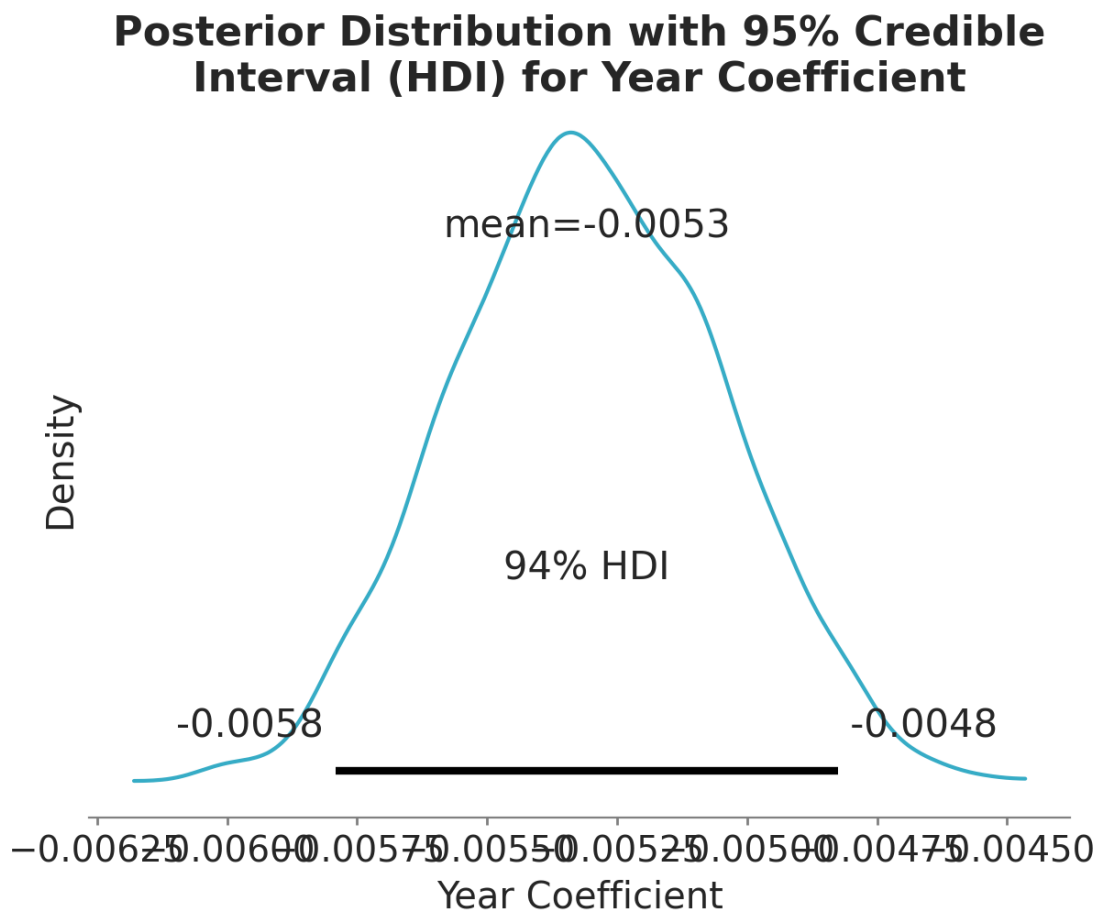
```
sns.kdeplot(year_samples, shade=True)
```



```
[7]: # Assuming `results` is the object returned after fitting the model
# Plotting the posterior for the 'year' coefficient (you can change 'year' to
      ↪ any other parameter)
az.plot_posterior(results, var_names=['year'])

# Customize the plot
plt.title('Posterior Distribution with 95% Credible Interval (HDI) for Year_
      ↪ Coefficient')
plt.xlabel('Year Coefficient')
plt.ylabel('Density')

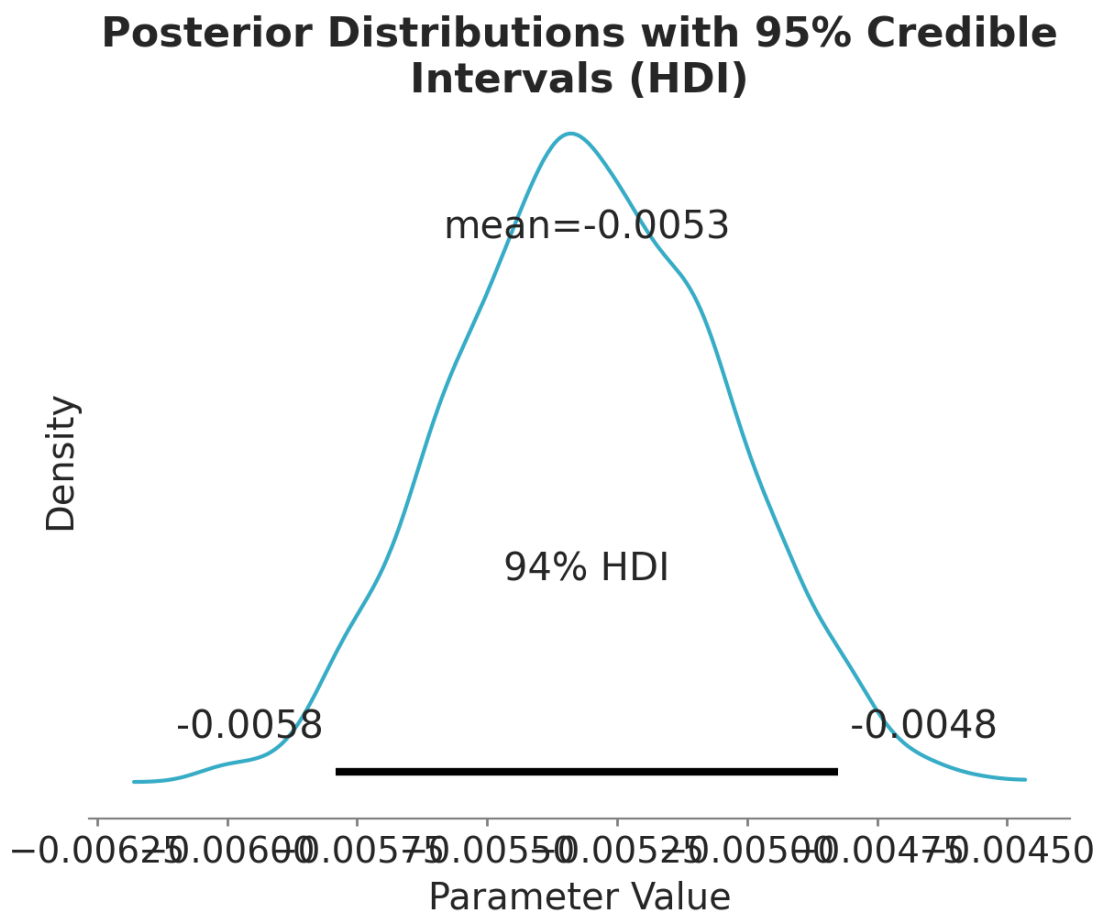
# Show the plot
plt.show()
```



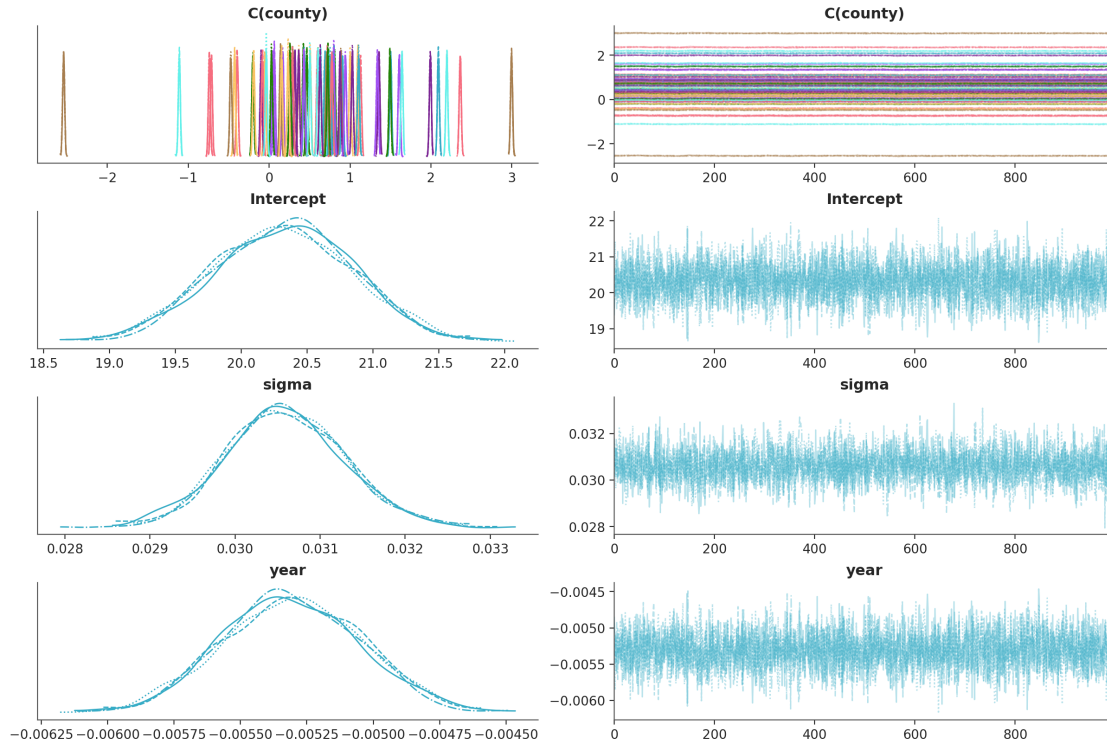
```
[8]: # Plotting the posterior distributions for the 'year' and 'county' coefficients
az.plot_posterior(results, var_names=['year'])

# Adding custom titles and labels
plt.title('Posterior Distributions with 95% Credible Intervals (HDI)')
plt.xlabel('Parameter Value')
plt.ylabel('Density')

# Show the plot
plt.show()
```



```
[9]: az.plot_trace(results);
```



```
[10]: az.summary(results)
```

```
[10]:
```

	mean	sd	hdi_3%	hdi_97%	mcse_mean	mcse_sd	\
C(county) [3.0]	0.782	0.012	0.761	0.806	0.001	0.000	
C(county) [5.0]	1.129	0.012	1.105	1.151	0.001	0.000	
C(county) [7.0]	0.369	0.012	0.346	0.393	0.001	0.000	
C(county) [9.0]	0.293	0.012	0.270	0.316	0.001	0.000	
C(county) [11.0]	0.422	0.012	0.400	0.445	0.001	0.000	
...	...	...	...	...	...	...	
C(county) [151.0]	0.631	0.012	0.608	0.653	0.001	0.000	
C(county) [153.0]	0.725	0.012	0.702	0.748	0.001	0.000	
Intercept	20.319	0.524	19.328	21.273	0.008	0.007	
sigma	0.031	0.001	0.029	0.032	0.000	0.000	
year	-0.005	0.000	-0.006	-0.005	0.000	0.000	

	ess_bulk	ess_tail	r_hat
C(county) [3.0]	135.0	531.0	1.02
C(county) [5.0]	150.0	551.0	1.02
C(county) [7.0]	145.0	498.0	1.02
C(county) [9.0]	143.0	509.0	1.02
C(county) [11.0]	150.0	516.0	1.02
...	...	...	...
C(county) [151.0]	150.0	597.0	1.02

C(county)[153.0]	140.0	614.0	1.02
Intercept	4094.0	3134.0	1.00
sigma	3413.0	2644.0	1.00
year	4108.0	3107.0	1.00

[80 rows x 9 columns]

```
[11]: model = bmb.Model("total_population ~ year", df)
      results = model.fit()
```

Initializing NUTS using jitter+adapt_diag...

Multiprocess sampling (4 chains in 4 jobs)

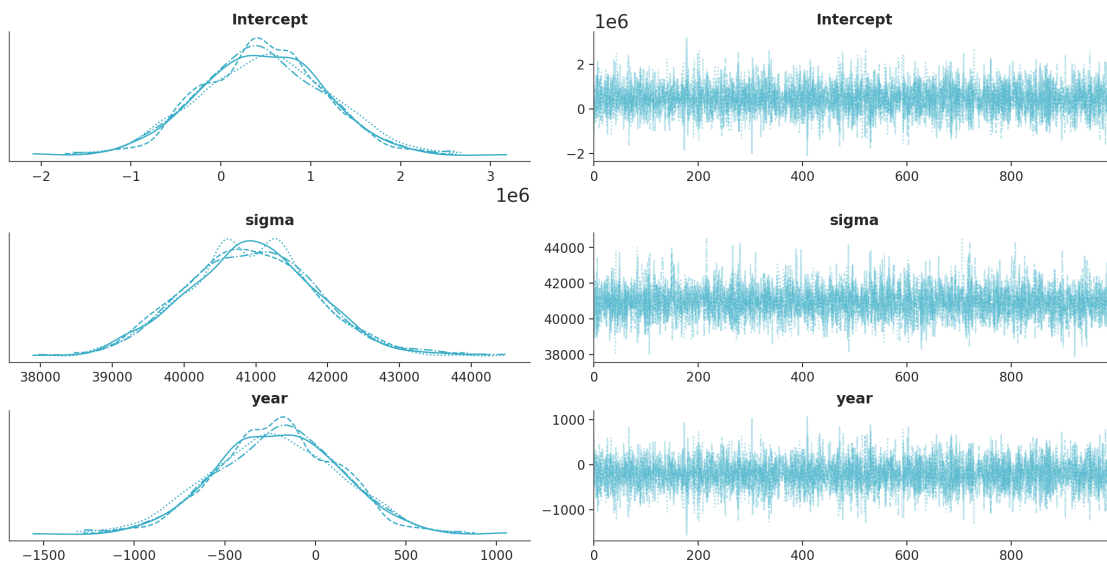
NUTS: [sigma, Intercept, year]

Output()

Sampling 4 chains for 1_000 tune and 1_000 draw iterations (4_000 + 4_000 draws total) took 1 seconds.

```
[12]: az.plot_trace(results)
```

```
[12]: array([[<Axes: title={'center': 'Intercept'}>,
             <Axes: title={'center': 'Intercept'}>],
            [<Axes: title={'center': 'sigma'}>,
             <Axes: title={'center': 'sigma'}>],
            [<Axes: title={'center': 'year'}>,
             <Axes: title={'center': 'year'}>]], dtype=object)
```




```
[13]: az.summary(results)
```

```
[13]:
```

	mean	sd	hdi_3%	hdi_97%	mcse_mean	\
Intercept	465983.817	696431.020	-786385.709	1795186.161	8977.995	
sigma	40931.891	909.181	39213.093	42574.632	12.000	
year	-212.965	345.280	-871.312	406.819	4.451	

	mcse_sd	ess_bulk	ess_tail	r_hat
Intercept	11460.123	6051.0	3093.0	1.0
sigma	15.050	5685.0	2967.0	1.0
year	5.684	6051.0	3093.0	1.0